



RUST PROGRAMMING

Introduction

Kamal kumar mukiri,
Apt Computing Labs



Outcome of this presentation:

- Course Logistics
- Importance of Rust Programming
 - Issues in C/C++
 - Core Features in Rust
- Installing Rust and Hands on
- Cargo for Project Management
 - Creating/Building/Running
 - Linting and Formatting
 - Writing/Running Test Cases
 - Getting Code Coverage Report
 - Documenting



Course logistics:

Course Logistics

Importance of Rust

Programming

Issues in C/C++

Core Features

Rust Installation

Project Management

Creating

Building

Running

Lint

Format

Unit Test Cases

Code Coverage

Document

All trainees will have access to **Daily recorded videos**, **Presentation slides** (can not be distributed), **Github path** where Assignments and practiced programs gets uploaded daily and **RoadMap of entire course**.

1. **Presentation slides:** (Google Drive)

https://drive.google.com/drive/folders/1-2OPLRbJDF8xWtocH8by7mX0eFUreLLk?usp=drive_link

2. **Daily recordings:**(Google Drive)

https://drive.google.com/drive/folders/1USDVOA_c6VUIl37VecXMbj2U4LZCNvYP?usp=drive_link

3. **Programs to practice for trainees and Assignments:** (GitHub)

<https://github.com/kamallearner123/APTRUSTSESS01/tree/main/Exercise>

4. **Course Roadmap:**

<https://roadmap.sh/r/aptrustsess01>



Course Logistics

Importance of Rust

Programming

Issues in C/C++

Core Features

Rust Installation

Project Management

Creating

Building

Running

Lint

Format

Unit Test Cases

Code Coverage

Document

Please check access to all the links



Books and References:

Course Logistics

Importance of Rust

Programming

Issues in C/C++

Core Features

Rust Installation

Project Management

Creating

Building

Running

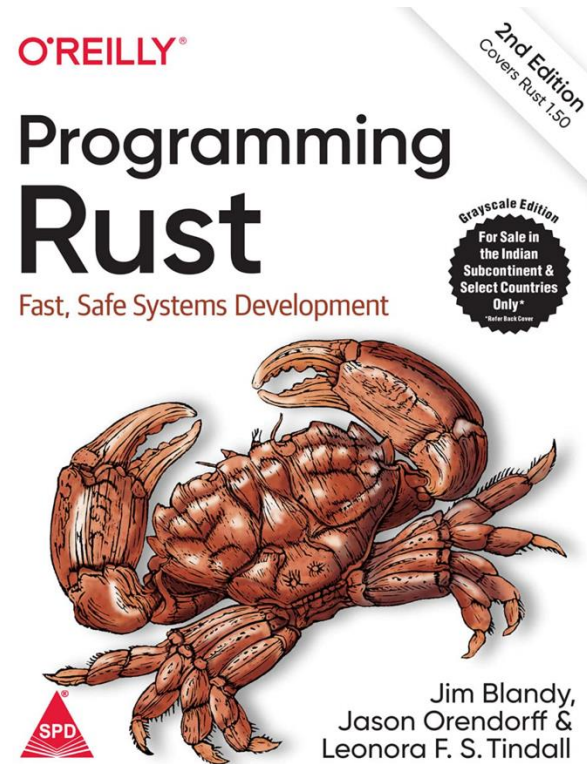
Lint

Format

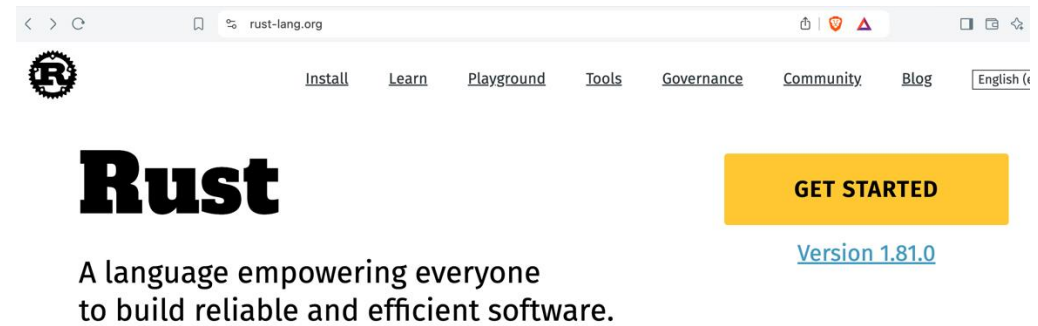
Unit Test Cases

Code Coverage

Document



Online tutorial and compiler: <https://www.rust-lang.org/>



Why Rust?

Performance

Rust is blazingly fast and memory-efficient: with no runtime or garbage collector, it can power performance-critical services, run on embedded devices, and easily integrate with other languages.

Reliability

Rust's rich type system and ownership model guarantee memory-safety and thread-safety — enabling you to eliminate many classes of bugs at compile-time.

Productivity

Rust has great documentation, a friendly compiler with useful error messages, and top-notch tooling — an integrated package manager and build tool, smart multi-editor support with auto-completion and type inspections, an auto-formatter, and more.

Supporting tools:

Course Logistics

Importance of Rust

Programming

Issues in C/C++

Core Features

Rust Installation

Project Management

Creating

Building

Running

Lint

Format

Unit Test Cases

Code Coverage

Document

The below are list of tools will be used as per need base.

- | | |
|-------------------------------|----------------------------------|
| 1. Debugging | : gdb, strace, lsof, top, etc... |
| 2. Memory/Multi-thread Issues | : Valgrind |
| 3. Performance | : perf |
| 4. Network monitor | : wireshark |
| 5. ELF file inspector | : readelf, ldd |
| 6. Unit test, coverage, doc | : cargo (in-built) |



Course Logistics

Importance of Rust

Programming

Issues in C/C++

Core Features

Rust Installation

Project Management

Creating

Building

Running

Lint

Format

Unit Test Cases

Code Coverage

Document

Market demand for Rust



Course Logistics

Importance of Rust
Programming

Issues in C/C++

Core Features

Rust Installation

Project Management

Creating

Building

Running

Lint

Format

Unit Test Cases

Code Coverage

Document

*“A language that doesn’t affect the way
you think about programming, is not
worth knowing”*

Alan Perlis



Course Logistics

Importance of Rust

Programming

Issues in C/C++

Core Features

Rust Installation

Project Management

Creating

Building

Running

Lint

Format

Unit Test Cases

Code Coverage

Document

Linux Torvald's comment on Rust Adaptation:

"I was expecting [Rust] updates to be faster, but part of the problem is that old-time kernel developers are used to C and don't know Rust," Torvalds said. "They're not exactly excited about having to learn a new language that is, in some respects, very different. So there's been some pushback on Rust." Torvalds added, however, that "another reason has been the Rust infrastructure itself has not been super stable."

<https://arstechnica.com/gadgets/2024/09/rust-in-linux-lead-retires-rather-than-deal-with-more-nontechnical-nonsense/>



Programming in Rust is like a **beautiful conversation** between a **programmer** and a **highly responsible yet strict compiler**.

- The communication is **clear**, with no room for ambiguity.
- Rust's compiler ensures that **assumptions**, which often lead to unexpected surprises in other languages, are **completely avoided**.
- It **demands** precision, guiding developers toward writing **safe and efficient code**, while still offering flexibility to craft performant software.



What type of language is Rust

Course Logistics

Importance of Rust

Programming

Issues in C/C++

Core Features

Rust Installation

Project Management

Creating

Building

Running

Lint

Format

Unit Test Cases

Code Coverage

Document

Coding languages are roughly classified into “levels”:
the lower the level, the less abstracted a language
is from the underlying hardware



Source: <https://kruschecompany.com/rust-language-concise-overview/>



Growth of Rust Programming

Course Logistics

Importance of Rust
Programming

Issues in C/C++

Core Features

Rust Installation

Project Management

Creating

Building

Running

Lint

Format

Unit Test Cases

Code Coverage

Document

GROWTH IN PROGRAMMING LANGUAGES 2021-2022

01 HCL	56.1%
02 Rust	50.5%
03 TypeScript	37.8%
04 Lua	34.2%
05 Go	28.3%
06 Shell	27.7%
07 Makefile	23.7%
08 C	23.5%
09 Kotlin	22.9%
10 Python	22.5%

Source: <https://octoverse.github.com/2022/top-programming-languages>



Rust is most loved programming Language for years

Course Logistics

Importance of Rust

Programming

Issues in C/C++

Core Features

Rust Installation

Project Management

Creating

Building

Running

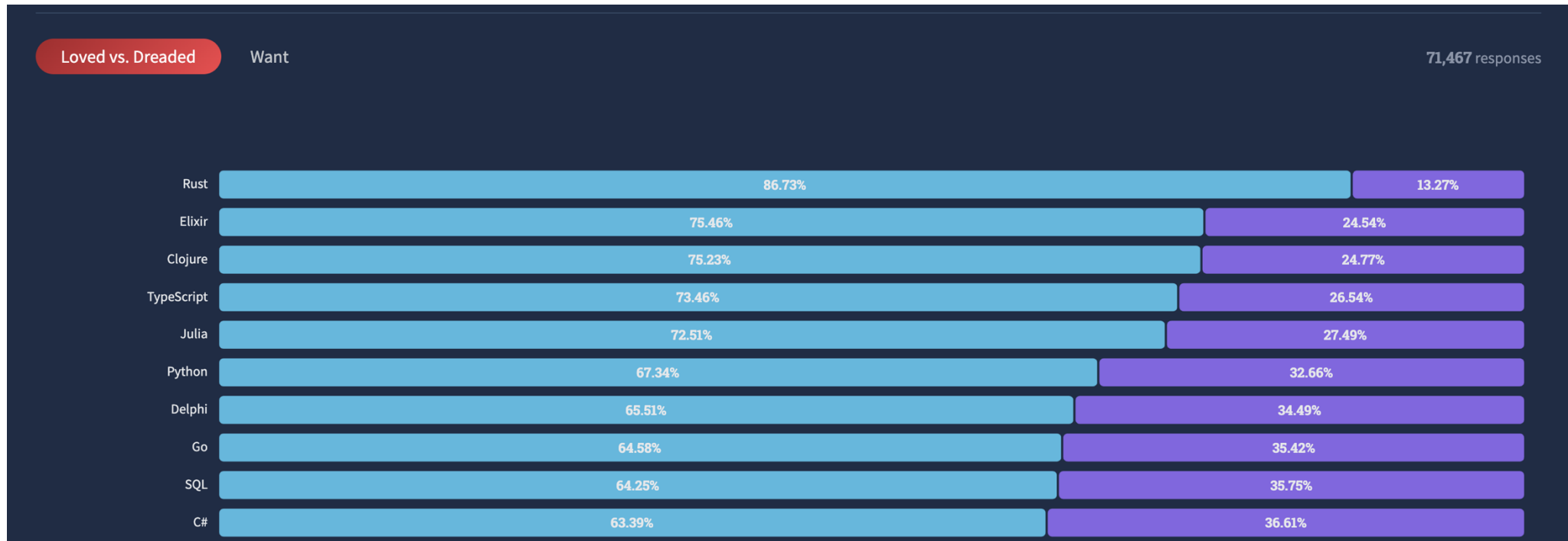
Lint

Format

Unit Test Cases

Code Coverage

Document



Source: https://survey.stackoverflow.co/2022/?utm_source=the+new+stack&utm_medium=referral&utm_content=inline-mention&utm_campaign=tns+platform#most-popular-technologies-language-prof



Course Logistics

Importance of Rust

Programming

Issues in C/C++

Core Features

Rust Installation

Project Management

Creating

Building

Running

Lint

Format

Unit Test Cases

Code Coverage

Document

1. Memory Safety Issues

- Dangling Pointers:** In C/C++, pointers to deallocated memory (dangling pointers) can cause crashes or unpredictable behavior if dereferenced.
- Use-After-Free:** After freeing memory, accessing it can lead to undefined behavior.
- Buffer Overflows:** Writing data beyond the allocated memory bounds can corrupt adjacent memory, leading to vulnerabilities (e.g., stack/heap overflow).
- Manual Memory Management:** Memory allocation and deallocation are error-prone, leading to potential memory leaks or over-releasing.
- Null Pointer Dereferencing:** Dereferencing a null pointer in C/C++ leads to crashes and is a common issue when pointers are not properly initialized or checked.



Course Logistics

Importance of Rust

Programming

Issues in C/C++

Core Features

Rust Installation

Project Management

Creating

Building

Running

Lint

Format

Unit Test Cases

Code Coverage

Document

2. Data Races in Multithreaded Programs

- Race Conditions:** C/C++ programs using threads are prone to data races where two threads access shared data concurrently, with at least one thread modifying the data.
- Manual Synchronization:** Developers need to use mutexes, locks, and other mechanisms, which are error-prone and can lead to deadlocks or incorrect synchronization.



Course Logistics

Importance of Rust

Programming

Issues in C/C++

Core Features

Rust Installation

Project Management

Creating

Building

Running

Lint

Format

Unit Test Cases

Code Coverage

Document

3. Undefined Behaviour

- **Undefined Behaviour (UB):** C/C++ has many undefined behaviours, such as integer overflows, dereferencing null pointers, or violating alignment rules.

These behaviours are not checked at compile time, potentially leading to serious bugs.

- **Type Casting:** Unsafe type casting in C/C++ (e.g., casting between incompatible types) can lead to undefined behaviour.



Course Logistics

Importance of Rust

Programming

Issues in C/C++

Core Features

Rust Installation

Project Management

Creating

Building

Running

Lint

Format

Unit Test Cases

Code Coverage

Document

4. Lack of Robust Error Handling

- **Error Handling:** In C/C++, error handling is often done through return codes, errno, or exceptions, leading to inconsistent and hard-to-maintain error-handling strategies.
- **Ignored Errors:** It's common for C/C++ developers to ignore or forget to check return values, leading to silent failures.



Course Logistics

Importance of Rust

Programming

Issues in C/C++

Core Features

Rust Installation

Project Management

Creating

Building

Running

Lint

Format

Unit Test Cases

Code Coverage

Document

5. Manual Resource Management

- **RAII (Resource Acquisition Is Initialization):** While C++ has RAII, developers need to manually implement it to manage resources like file handles and sockets, and there's no equivalent in C.
- **Leaks:** Forgetting to free resources leads to memory and resource leaks.



Course Logistics

Importance of Rust

Programming

Issues in C/C++

Core Features

Rust Installation

Project Management

Creating

Building

Running

Lint

Format

Unit Test Cases

Code Coverage

Document

4. Lack of Robust Error Handling

- Error Handling:** In C/C++, error handling is often done through return codes, errno, or exceptions, leading to inconsistent and hard-to-maintain error-handling strategies.
- Ignored Errors:** It's common for C/C++ developers to ignore or forget to check return values, leading to silent failures.



Core features provided by Rust

Course Logistics

Importance of Rust

Programming

Issues in C/C++

Core Features

Rust Installation

Project Management

Creating

Building

Running

Lint

Format

Unit Test Cases

Code Coverage

Document

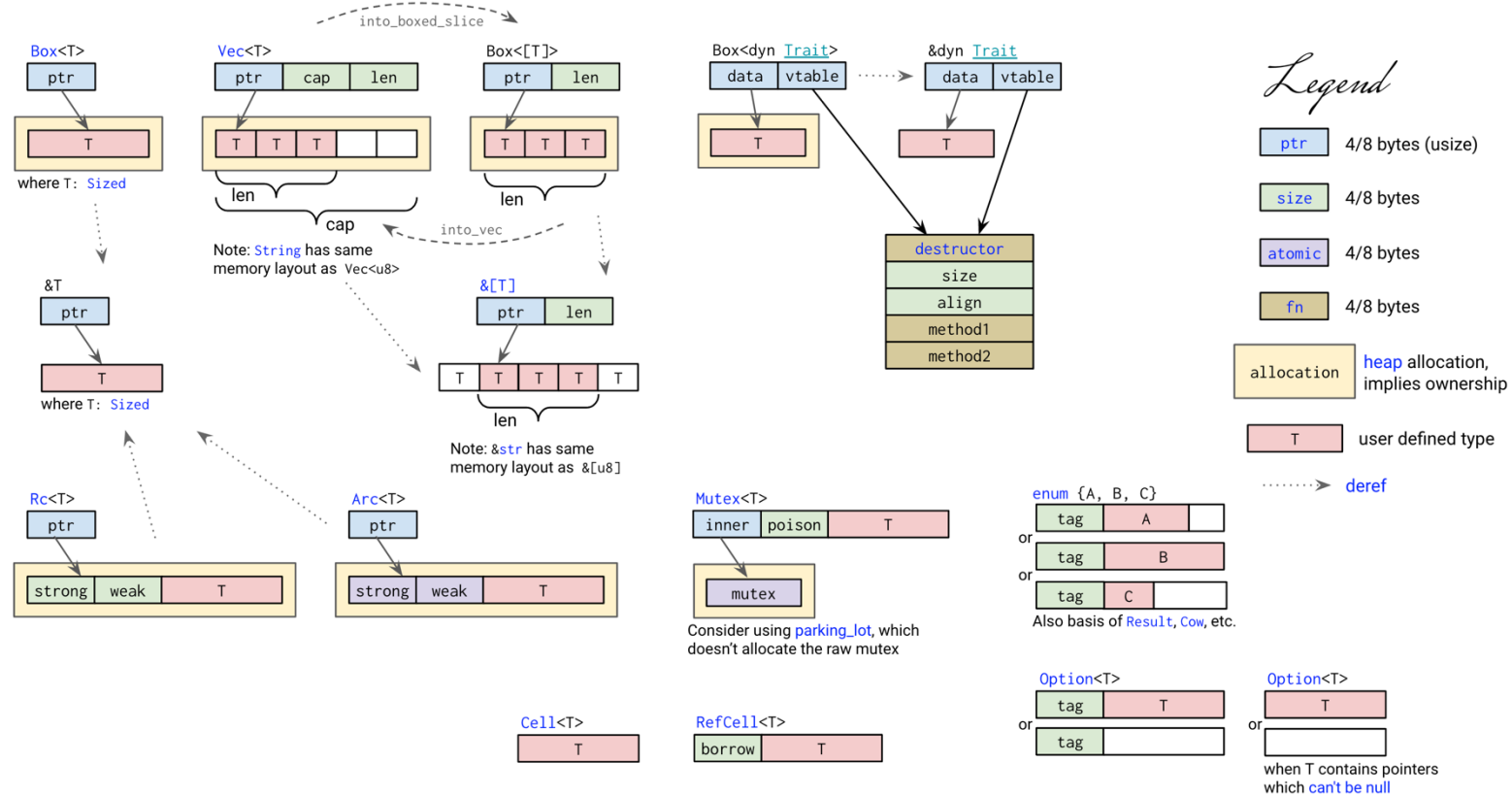
- Ownership and Borrowing
- Lifetimes
- Pattern Matching
- Zero-Cost Abstractions
- Memory Safety without Garbage Collection
- Traits and Trait Objects
- Concurrency without Data Races
- Algebraic Data Types (Enums and Structs)
- Error Handling with Result and Option
- Type Inference
- Macros (Declarative and Procedural)
- Immutable and Mutable Variables
- Crate and Module System
- Closures and Functional Programming Features
- Asynchronous Programming (Async/Await)
- Safe and Unsafe Code Blocks
- Iterator and Iterator Adapters
- Smart Pointers (Box, Rc, Arc)
- Embedded Programming Support
- Cross-Compilation

All the features will be discussed in this source in deep with examples.



Built-in data types

Rust Container cheat sheet:



Rust container cheat sheet, by Raph Levien, Copyright 2017 Google Inc., released under Creative Commons BY, 2017-04-21, version 0.0.4

Source: https://docs.google.com/presentation/d/1q-c7UAyrUIM-eZyTo1pd8SZ0qwA_wYxmPZVOQkoDmH4/edit#slide=id.p



Course Logistics

Importance of Rust

Programming

Issues in C/C++

Core Features

Rust Installation

Project Management

Creating

Building

Running

Lint

Format

Unit Test Cases

Code Coverage

Document

Practice Time



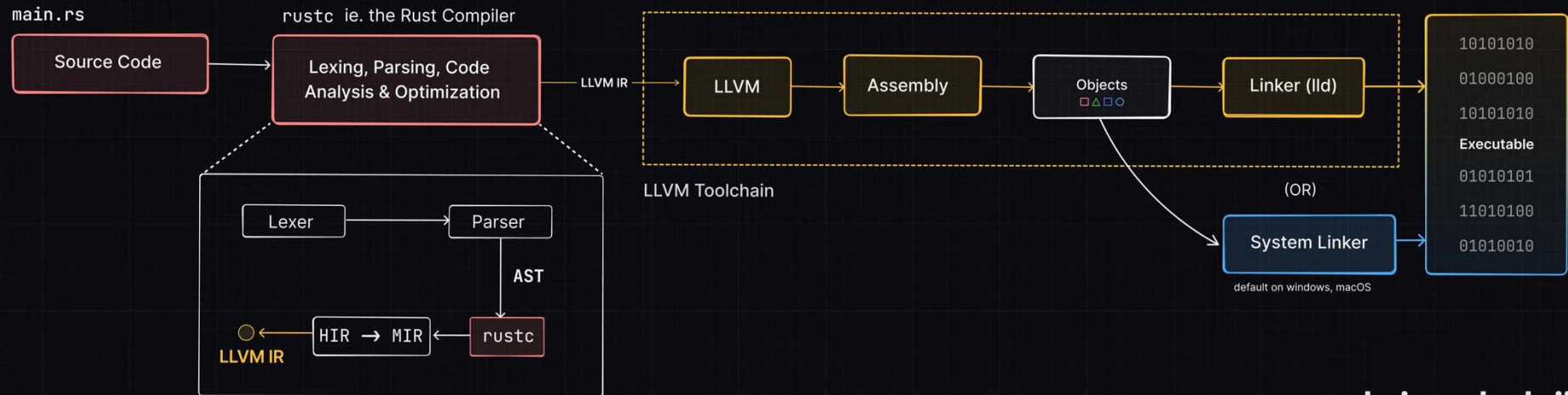
- Course Logistics
- Importance of Rust
- Programming
 - Issues in C/C++
 - Core Features
- Rust Installation
- Project Management
 - Creating
 - Building
 - Running
 - Lint
 - Format
 - Unit Test Cases
 - Code Coverage
 - Document

Issues	Rust Features
Stacks, dangling pointers	Ownership and Borrowing System
Errors	Option<T> for safe null handling
Control flows	Bounds checking on arrays and slices
Memory free	Ownership, Lifetimes, and Borrowing
Automatic memory management	Automatic memory management via RAI (Resource Acquisition Is Initialization)
Concurrency	Concurrency with Ownership, Send, Sync, Arc
No pointer arithmetic	No pointer arithmetic in safe Rust
Error handling	Result<T, E> and pattern matching
Compiler errors	Rust's clear and detailed compiler messages
Safe issues	Safe concurrency primitives (Mutex, Arc)
Build systems	Cargo package manager and build system

Rust compilation process:

Resolving Rust Symbols

Compilation, Linkers & Visualizing Symbols



@shrirambalaji

Source: <https://blog.shrirambalaji.com/posts/resolving-rust-symbols/>

Apt Computing Labs



Course Logistics

Importance of Rust

Programming

Issues in C/C++

Core Features

Rust Installation

Project Management

Creating

Building

Running

Lint

Format

Unit Test Cases

Code Coverage

Document

Course Logistics

Importance of Rust

Programming

Issues in C/C++

Core Features

Rust Installation

Project Management

Creating

Building

Running

Lint

Format

Unit Test Cases

Code Coverage

Document

Cargo: Build and Package System



Rust Core tools:

Course Logistics

Importance of Rust

Programming

Issues in C/C++

Core Features

Rust Installation

Project Management

Creating

Building

Running

Lint

Format

Unit Test Cases

Code Coverage

Document

Rustup: This tool helps manage Rust versions, a crucial feature as Rust is continually evolving to meet the demands of high-performance computing.

Cargo: Rust's dependency management tool relies on TOML configuration files. Cargo is also the main frontend for the compilation process – Simply type **cargo run** or **cargo build --release**. With Cargo Watch, developers can leverage the most innovative iterative workflow.

TOML (Tom's Obvious, Minimal Language): It is a configuration file format commonly used for specifying settings in Rust projects. It's the central place where all dependencies are defined.

Rustc: Rust's compiler isn't just a compiler; it's also a static code checker and transpiler. It generates intermediate code passed to LLVM for optimization, a critical aspect of making applications efficient and performant.



Cargo: Tools

Course Logistics

Importance of Rust

Programming

Issues in C/C++

Core Features

Rust Installation

Project Management

Creating

Building

Running

Lint

Format

Unit Test Cases

Code Coverage

Document

- `cargo build` - Compiles the current project.
-
- `cargo run` - Compiles and runs the project.
-
- `cargo test` - Runs all tests in the project.
-
- `cargo fmt` - Formats the project's code according to Rust's style guidelines.
-
- `cargo clippy` - Runs the Clippy linter to catch common mistakes and improve code quality.
-
- `cargo check` - Quickly checks the code for errors without generating an executable.
-
- `cargo doc` - Generates documentation for the project.
-
- `cargo install <package>` - Installs a Rust binary package.
-
- `cargo add <dependency>` - Adds a dependency to Cargo.toml (via `cargo-edit`).



Cargo: Tools

Course Logistics

Importance of Rust

Programming

Issues in C/C++

Core Features

Rust Installation

Project Management

Creating

Building

Running

Lint

Format

Unit Test Cases

Code Coverage

Document

- `cargo rm <dependency>` - Removes a dependency from Cargo.toml (via `cargo-edit`).
- `cargo update` - Updates dependencies in Cargo.lock.
- `cargo outdated` - Displays outdated dependencies (via `cargo-outdated`).
- `cargo audit` - Audits the project's dependencies for known security vulnerabilities (via `cargo-audit`).
- `cargo bench` - Runs benchmarks in the project.
- `cargo watch -x <command>` - Watches for changes and runs a command when files are modified (via `cargo-watch`).
- `cargo tarpaulin` - Runs code coverage analysis (via `cargo-tarpaulin`).
- `cargo release` - Automates the process of creating and managing project releases (via `cargo-release`).
- `cargo make` - Defines and automates custom tasks in a project (via `cargo-make`).
- `cargo generate` - Creates a new project from a template (via `cargo-generate`).
- `cargo generate --git <template-repo-url>`
- `cargo llvm-lines` - Identifies large functions in terms of generated LLVM IR (via `cargo-llvm-lines`).
- `cargo clean` - Removes the target directory to free up disk space or force a rebuild.



Course Logistics

Importance of Rust

Programming

Issues in C/C++

Core Features

Rust Installation

Project Management

Creating

Building

Running

Lint

Format

Unit Test Cases

Code Coverage

Document

Creating a project using cargo



Rust tool chain and echo system

Course Logistics

Importance of Rust

Programming

Issues in C/C++



Core Features

Rust Installation

Project Management

Creating

Building

Running

Lint

Format

Unit Test Cases

Code Coverage

Document

- 1) Creating a project
- 2) Examine Cargo.toml
- 3) Adding additional Binaries
- 4) Adding additional Libraris



Creating project using “Cargo”:

Course Logistics

Importance of Rust

Programming

Issues in C/C++

Core Features

Rust Installation

Project Management

Creating

Building

Running

Lint

Format

Unit Test Cases

Code Coverage

Document

✓ IDS

✓ src ①

⚙️ main.rs

≡ Cargo.lock ②

⚙️ Cargo.toml ③

≡ Cargo.lock

```
1 # This file is automatically @generated by Cargo.
2 # It is not intended for manual editing.
3 version = 3
4
5 [[package]]
6 name = "ids"
7 version = "0.1.0"
8
```

⚙️ Cargo.toml

```
1 [package]
2 name = "ids"
3 version = "0.1.0"
4 edition = "2021"
5
6 [dependencies]
7
```

Creating a project (Target is binary)

```
% cargo new ids
```

- 1) **src**: Source directory (main.rs should be there). All the files should be extended with .rs
- 2) **Cargo.lock** freezes the state of dependencies, allowing for consistent builds across different environments and times.
- 3) Cargo.toml keeps the information about package and its dependencies. It can be edited manually unlike “Cargo.lock”



Building project using “Cargo”:

Course Logistics

Importance of Rust

Programming

Issues in C/C++

Core Features

Rust Installation

Project Management

Creating

Building

Running

Lint

Format

Unit Test Cases

Code Coverage

Document

▼ IDS

▼ src

🔗 ids_filter.rs

🔗 ids_idsm.rs

🔗 main.rs

▼ target

▼ debug

> .fingerprint

1 > build

> deps 2

▼ examples 3

4 > incremental

≡ .cargo-lock

5 ≡ ids

6 D ids.d

7 ≡ idsm

8 D idsm.d

9 D libids_filter.d

≡ libids_filter.rlib 10

{ } .rustc_info.json

≡ CACHEDIR.TAG

≡ Cargo.lock

⚙ Cargo.toml

Building project

```
% cargo build
```



Adding binaries/libraries using “Cargo”:

Course Logistics

Importance of Rust

Programming

Issues in C/C++

Core Features

Rust Installation

Project Management

Creating

Building

Running

Lint

Format

Unit Test Cases

Code Coverage

Document

▼ IDS

▼ src

📄 main.rs

▼ target

▼ debug

> .fingerprint

> build

> deps

> examples

> incremental

≡ .cargo-lock

≡ ids

📄 ids.d

{ } .rustc_info.json

≡ CACHEDIR.TAG

≡ Cargo.lock

⚙️ Cargo.toml

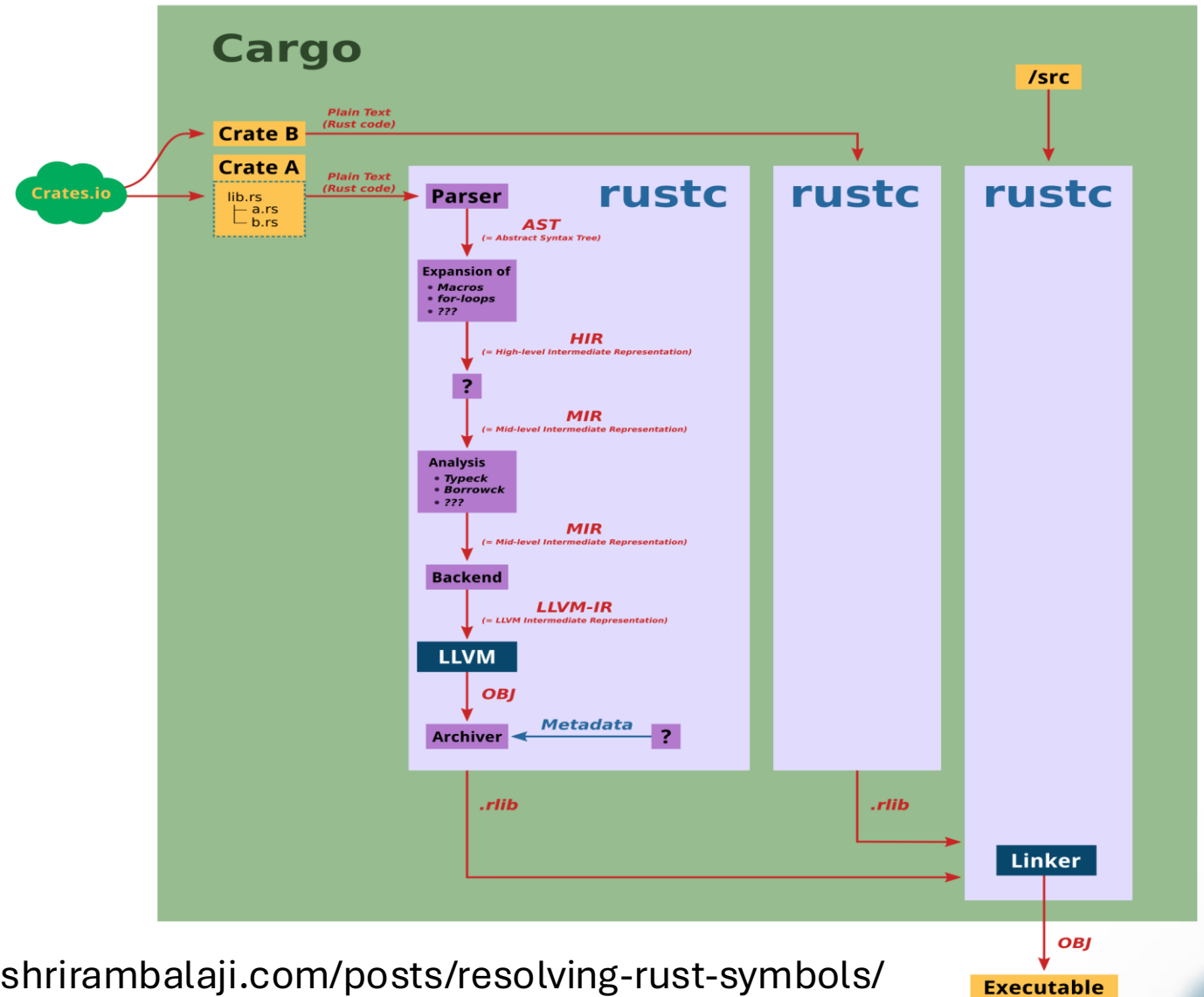
Creating a project (Target is binary)

```
% cargo build
```

1) “ids”: By default, debug binary gets created



Cargo build system:

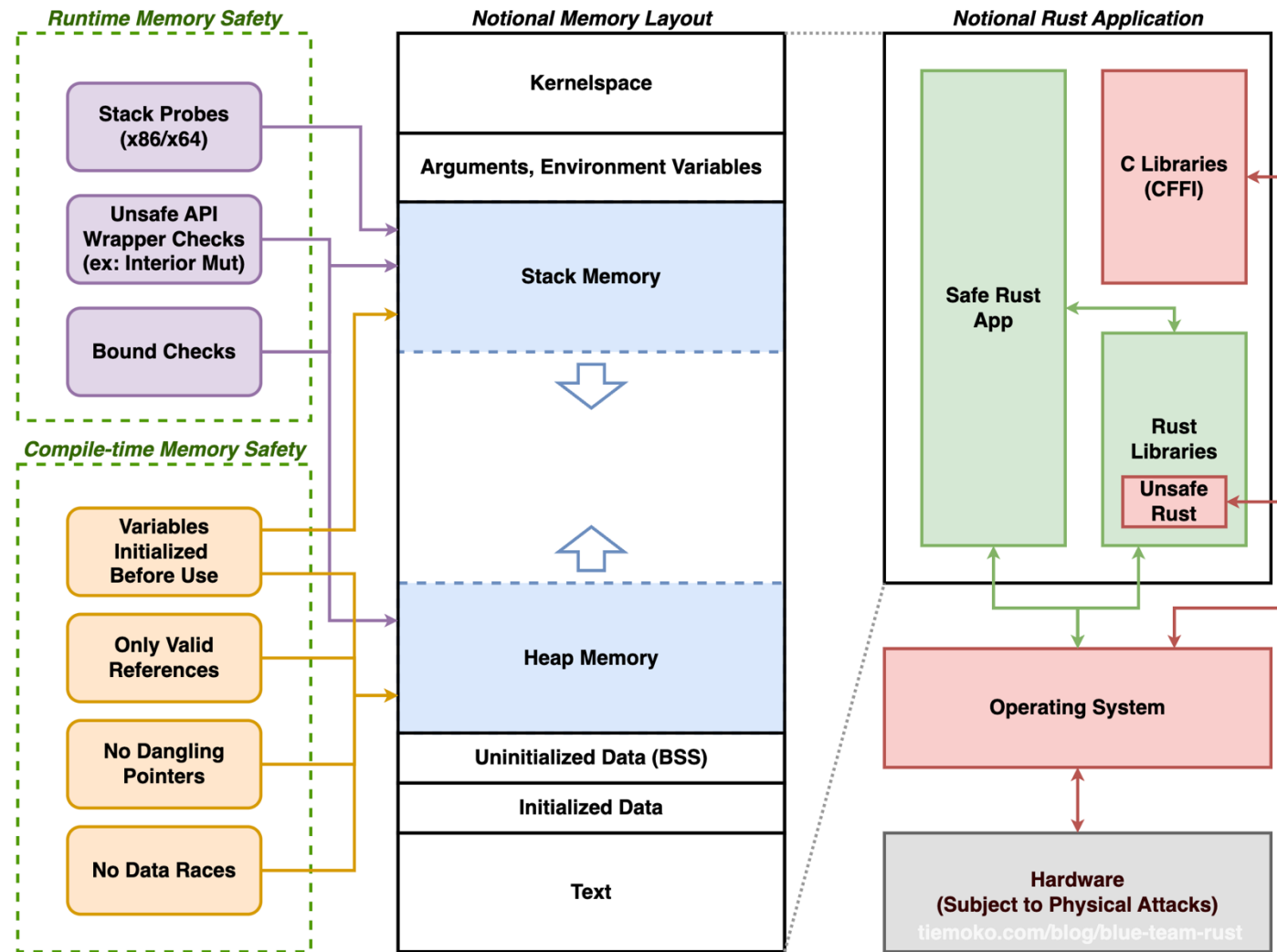


Source: <https://blog.shrirambalaji.com/posts/resolving-rust-symbols/>



Process Memory and Features

- Course Logistics
- Importance of Rust
- Programming
 - Issues in C/C++
 - Core Features
- Rust Installation
- Project Management
 - Creating
 - Building
 - Running
 - Lint
 - Format
 - Unit Test Cases
 - Code Coverage
 - Document



Source: <https://tiemoko.com/blog/blue-team-rust/>



Course Logistics

Importance of Rust

Programming

Issues in C/C++

Core Features

Rust Installation

Project Management

Creating

Building

Running

Lint

Format

Unit Test Cases

Code Coverage

Document

Clippy tool



\$cargo clippy

Course Logistics

Importance of Rust

Programming

Issues in C/C++

Core Features

Rust Installation

Project Management

Creating

Building

Running

Lint

Format

Unit Test Cases

Code Coverage

Document

A lint tool, does not generate binary/library.

Finds issues:

- 1) Style
- 2) Correction
- 3) Complexity
- 4) Performance



\$cargo clippy

Course Logistics

Importance of Rust

Programming

Issues in C/C++

Core Features

Rust Installation

Project Management

Creating

Building

Running

Lint

Format

Unit Test Cases

Code Coverage

Document

Sample errors:

- 1) No need of return
- 2) Useless code
- 3) Too many args
- 4) Iterator count



Course Logistics

Importance of Rust

Programming

Issues in C/C++

Core Features

Rust Installation

Project Management

Creating

Building

Running

Lint

Format

Unit Test Cases

Code Coverage

Document

Rust format tool



\$cargo fmt

Course Logistics

Importance of Rust

Programming

Issues in C/C++

Core Features

Rust Installation

Project Management

Creating

Building

Running

Lint

Format

Unit Test Cases

Code Coverage

Document

```
3 fn fun() -> u8 {return 0;}
```

```
4
```

```
5 fn main() {  
6 println!("Hello, world!");  
7 }  
8
```

```
3 fn fun() -> u8 {
```

```
4 |
```

```
    return 0;
```

```
5 }
```

```
6
```

```
7 fn main() {  
8 |    println!("Hello, world!");  
9 }
```

1. Code before running \$cargo fmt
2. Code after running command \$cargo fmt



Course Logistics

Importance of Rust

Programming

Issues in C/C++

Core Features

Rust Installation

Project Management

Creating

Building

Running

Lint

Format

Unit Test Cases

Code Coverage

Document

Testing code



Testing code: Writing test case

Course Logistics

Importance of Rust

Programming

Issues in C/C++

Core Features

Rust Installation

Project Management

Creating

Building

Running

Lint

Format

Unit Test Cases

Code Coverage

Document

```
1  mod mymod;
2
3  fn fun() -> u8 {
4      |   return 0;
5      |
6  }
7  fn main() {
8      |   println!("Hello, world!");
9      |
10 }
11
12 #[cfg(test)] ①
13 mod test {
14     |
15     use super::*; ②
16     |
17     #[test]
18     fn test_01() {
19         |   assert_eq!(fun(), 0);
20         |
21     }
22
23     #[test]
24     fn test_mymod() {
25         |   assert_eq!(mymod::whole_square(1,2), 9);
26         |
27     }
28 }
29
```

```
③ kamalmukiri@Kamals-MacBook-Air:~/Documents/GitHub/Courses/APTRUSTSESS01/Exercise/Kamal/3.IdiomaticCode/fmt_clippy$ cargo test
Compiling fmt_clippy v0.1.0 (/Users/kamalmukiri/Documents/GitHub/Courses/APTRUSTSESS01/Exercise/Kamal/3.IdiomaticCode/fmt_clippy)
Finished `test` profile [unoptimized + debuginfo] target(s) in 0.88s
Running unittests src/main.rs (target/debug/deps/fmt_clippy-ab8821f7788ca832)

running 2 tests
test test::test_01 ... ok
test test::test_mymod ... ok

test result: ok. 2 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s
```

1. `#[cfg(test)]` attribute is used to conditionally include code only when compiling for tests
2. `#[test]` Marks a function as a test case.
3. `$cargo test`: Running test cases



\$cargo test: Toolsß

Course Logistics

Importance of Rust

Programming

Issues in C/C++

Core Features

Rust Installation

Project Management

Creating

Building

Running

Lint

Format

Unit Test Cases

Code Coverage

Document

Prerequisites:

```
$cargo install cargo-tarpaulin
```

```
$cargo install cargo-nextest
```

Running tools:

```
$ cargo tarpaulin --out Html      (To generate coverage report)
```

```
$ cargo nextest run --test-threads=2 test --message-format=libtest-  
json > TestReport.json
```

Usage: cargo nextest run [OPTIONS] [FILTERS]... [-- <FILTERS_AND_ARGS>...]

Usage: cargo nextest run [OPTIONS] [FILTERS]... [-- <FILTERS_AND_ARGS>...]



\$cargo test: Coverage

Course Logistics

Importance of Rust

Programming

Issues in C/C++

Core Features

Rust Installation

Project Management

Creating

Building

Running

Lint

Format

Unit Test Cases

Code Coverage

Document

Find the report in main directory:

```
▼ fmt_clippy
  ▼ src
    Ⓢ main.rs
  > target
  ≡ Cargo.lock
  ⚙ Cargo.toml
  <> tarpaulin-report.html
```

Test report:

/Users/kamalmukiri/Documents/GitHub/Courses/APTRUSTSESS01/Exercise/Kamal/3.IdiomaticCode/fmt_clippy/src		Covered: 3 of 5 (60.00%)
Path	Coverage	
main.rs	2 / 4 (50.00%)	
mymod.rs	1 / 1 (100.00%)	

Back /Users/kamalmukiri/Documents/GitHub/Courses/APTRUSTSESS01/Exercise/Kamal/3.IdiomaticCode/fmt_clippy/src/main.rs	Covered: 2 of 4 (50.00%)
<pre>1 mod mymod; 2 3 fn fun() -> u8 { 4 return 0; 5 } 6 fn main() { 7 println!("Hello, world!"); 8 } 9 10 #[cfg(test)] 11 mod test { 12 use super::*; 13 #[test] 14 fn test_01() { 15 assert_eq!(fun(), 0); 16 } 17 18 #[test] 19 fn test_mymod() { 20 assert_eq!(mymod::whole_square(1,2), 9); 21 } 22 } 23</pre>	



Course Logistics

Importance of Rust

Programming

Issues in C/C++

Core Features

Rust Installation

Project Management

Creating

Building

Running

Lint

Format

Unit Test Cases

Code Coverage

Document

Documenting code



Documenting: Outer documentation (///)

Course Logistics

Importance of Rust

Programming

Issues in C/C++

Core Features

Rust Installation

Project Management

Creating

Building

Running

Lint

Format

Unit Test Cases

Code Coverage

Document

```
7  /// # History ①
8  /// areaCircle: Function which calculates area of circle and returns
9  ② /// - r : Radius in f32
10 /// - Returns: Area in f32
11 /// ## References
12 /// - Please refer [whole_square](mymod::whole_square) ③
13 fn areaCircle(r:f32) -> f32 {
14     |   return PI*r.powf(2.0);
15 }
```

Output

```
④ kamalmukiri@Kamals-MacBook-Air clippy_fmt % cargo doc --no-deps --open
```

Comments can be given with “///” as prefix.

1. **#History**: Adding headings
2. **- r**: Adding bullets.
3. **[`Refer``](link)**: Hyper link
“whole_square” is link name,
“mymod::whole_square” is link.
4. **Generating** document
“—no-deps”: No need to generate doc for dependencies
“—open”: Open the document as soon as the command is executed.



Documenting

- Course Logistics
- Importance of Rust Programming
 - Issues in C/C++
 - Core Features
- Rust Installation
- Project Management
 - Creating
 - Building
 - Running
 - Lint
 - Format
 - Unit Test Cases
 - Code Coverage
 - Document**

clippy_fmt 0.1.0

Type 'S' or '/' to search, '?' for more options...

Crate **clippy_fmt**

source · [-]

Modules

mymod

Constants

PI

Functions

areaCircle History
main This is main function.

clippy_fmt 0.1.0

Type 'S' or '/' to search, '?' for more options...

Function **clippy_fmt::areaCircle**

source · [-]

pub(crate) fn areaCircle(r: f32) -> f32

History

areaCircle: Function which calculates area of circle and returns

- r : Radius in f32
- Returns: Area in f32

References

- main main
- Please refer whole_square



Documenting: Inner documentation (//!)

Course Logistics

Importance of Rust

Programming

Issues in C/C++

Core Features

Rust Installation

Project Management

Creating

Building

Running

Lint

Format

Unit Test Cases

Code Coverage

Document

{

least 3 arguments are required.

radius:f32 = (10.0);

_vec:Vec<i32> = vec![1,2,3,4];

n!("Radius = {}", radius);

n!("Area = {}",areaCircle(radius));

```
als-MacBook-Air clippy_fmt % cargo doc --no-deps --open
```

Inner comments can be given with “//!” with prefix.



Documenting:

Course Logistics

Importance of Rust

Programming

Issues in C/C++

Core Features

Rust Installation

Project Management

Creating

Building

Running

Lint

Format

Unit Test Cases

Code Coverage

Document

All mark down features are supported.

More Options: <https://doc.rust-lang.org/cargo/commands/cargo-doc.html>



Thank you !!!

